

Graph Implementation

Directed & Weighted Directed Graphs

C++17 | Adjacency List | STL Vectors

Course	Data Structures and Problem Solving
Assignment	Q50
Student	Youssef Amr Abdelrahman
Student ID	23P0257
University	Ain Shams University
Language	C++17

This report covers two graph implementations using adjacency lists in C++17: a directed graph and a weighted directed graph — including full source code, output, and complexity analysis.

1. Problem Statement

Implement a **graph** data structure using an **adjacency list** in two forms. Both use directed edges — a connection from A to B does **not** imply B to A:

- **Directed Graph** — edges connect nodes with direction but no cost.
- **Weighted Directed Graph** — edges have both direction and a weight value.

Total nodes	7 (labelled 0 – 6)
Total edges	8
Graph type	Directed (both implementations)
Weights	None (directed) / Integer (weighted directed)

2. Directed Graph

Each edge {src, dest} creates a one-way connection. The adjacency list stores a **vector of ints** per node.

Graph Structure

```
Edges: {0,2}, {0,3}, {1,3}, {1,4}, {2,5}, {3,5}, {4,5}, {4,6}
0 --> 2 3      3 --> 5
1 --> 3 4      4 --> 5 6
2 --> 5        5 --> (none)  6 --> (none)
```

Source Code

```
#include <iostream>
#include <vector>
using namespace std;
struct Edge { int src, dest; };
class Graph {
public:
    vector<vector<int>> adjList;
    Graph(vector<Edge> const &edges, int n) {
        adjList.resize(n);
        for (const auto &edge : edges)
            adjList[edge.src].push_back(edge.dest);
    }
};

void printGraph(Graph const &graph, int n) {
    for (int i = 0; i < n; i++) {
        cout << i << ' --> ';
        for (int v : graph.adjList[i]) cout << v << ' ';
        cout << endl;
    }
}

int main() {
    vector<Edge> edges = {{0,2},{0,3},{1,3},{1,4},{2,5},{3,5},{4,5},{4,6}};
    Graph graph(edges, 7);
    printGraph(graph, 7);
    return 0;
}
```

```
}
```

Output

```
0 --> 2 3
1 --> 3 4
2 --> 5
3 --> 5
4 --> 5 6
5 -->
6 -->
```

3. Weighted Directed Graph

Each edge **{src, dest, weight}** stores a cost alongside the direction. The adjacency list stores **pairs (destination, weight)** — making this structure directly usable by Dijkstra's or Bellman-Ford algorithms.

Graph Structure

```
Edges: {0,2,9},{0,3,2},{1,3,4},{1,4,7},{2,5,8},{3,5,6},{4,5,5},{4,6,3}
(0->2, w=9) (0->3, w=2) (3->5, w=6)
(1->3, w=4) (1->4, w=7) (4->5, w=5) (4->6, w=3)
(2->5, w=8) 5 and 6 have no outgoing edges
```

Source Code

```
#include <iostream>
#include <vector>
using namespace std;
struct Edge { int src, dest, weight; };
typedef pair<int,int> Pair;
class Graph {
public:
    vector<vector<Pair>> adjList;
    Graph(vector<Edge> const &edges, int n) {
        adjList.resize(n);
        for (auto &e : edges)
            adjList[e.src].push_back(make_pair(e.dest, e.weight));
    }
};

void printGraph(Graph const &graph, int n) {
    for (int i = 0; i < n; i++) {
        for (Pair v : graph.adjList[i])
            cout << '(' << i << ', ' << v.first << ', ' << v.second << ') ' ;
        cout << endl;
    }
}

int main() {
    vector<Edge> edges = {
        {0,2,9},{0,3,2},{1,3,4},{1,4,7},{2,5,8},{3,5,6},{4,5,5},{4,6,3}};
    Graph graph(edges, 7);
    printGraph(graph, 7);
    return 0;
}
```

```
}
```

Output

```
(0, 2, 9) (0, 3, 2)
(1, 3, 4) (1, 4, 7)
(2, 5, 8)
(3, 5, 6)
(4, 5, 5) (4, 6, 3)
```

4. Complexity Analysis

Construction	$O(E)$ — each edge inserted once into the adjacency list
Printing	$O(V + E)$ — every node and edge visited once
Space	$O(V + E)$ — adjacency list stores all nodes and edges

5. Comparison

Feature	Directed Graph	Weighted Directed
Edge struct	{src, dest}	{src, dest, weight}
Adjacency list	vector<vector<int>>	vector<vector<pair>>
Time complexity	$O(V + E)$	$O(V + E)$
Space complexity	$O(V + E)$	$O(V + E)$
Supports weights	No	Yes
Use cases	DAGs, dependencies	Road maps, routing

Key Takeaways

- Both use **adjacency list** representation — compact and efficient for sparse graphs.
- The weighted version extends the directed graph by storing weights as **pairs**.
- The weighted structure is **ready for Dijkstra's / Bellman-Ford** algorithms.
- Both handle **directed edges** only — no reverse edges added.

Real-World Use Cases

Directed Graph	Task dependency graphs, compiler DAGs, web link graphs
Weighted Directed Graph	Road navigation, network routing, resource allocation